DATA WAREHOUSE DE VENDAS DO ADVENTUREWORKS: MODELAGEM ESTRELA E ETL INCREMENTAL ORQUESTRADO COM APACHE AIRFLOW

Lucas de Lima Nascimento
James Alves
Centro Universitário Salesiano - UniSales
Curso: Sistemas de Informação

RESUMO

Este artigo apresenta o projeto e a implementação de um *Data Warehouse* (DW) de vendas construído a partir do banco transacional AdventureWorks2016. O modelo multidimensional segue o padrão estrela, com uma tabela fato no grão de item de pedido e seis dimensões, implementado em PostgreSQL. O processo de extração, transformação e carga (ETL) usa uma área de *stage*: a origem é lida apenas em extrações curtas das chaves alteradas, detectadas pelo *Change Tracking* do SQL Server, e toda a transformação ocorre depois, em *Structured Query Language* (SQL), dentro do DW. O Apache Airflow orquestra o processo em seis blocos com 17 tarefas, muitas em paralelo, e dez indicadores foram implementados como *views* SQL. Em um teste controlado, alterações na origem levaram a execução incremental a ler somente 1 dos 31.465 pedidos, 1 produto e 2 clientes. Os indicadores apontam receita líquida de US$ 109,8 milhões, concentração de 86,2% da receita na categoria *Bikes* e forte oscilação mensal provocada pelo canal de revendedores.

Palavras-chave: *data warehouse*; modelo estrela; ETL incremental; área de *stage*; Apache Airflow.

ABSTRACT

This paper presents the design and implementation of a sales data warehouse built from the AdventureWorks2016 transactional database. The multidimensional model follows the star schema, with a fact table at order-line grain and six dimensions, implemented in PostgreSQL. The extract, transform and load (ETL) process uses a staging area: the source is read only in short extractions of the changed keys, detected by SQL Server Change Tracking, and every transformation runs later, in SQL, inside the warehouse. Orchestration in Apache Airflow splits the process into six blocks with 17 tasks, many running in parallel, and each group of tables keeps its own checkpoint. Ten business indicators were implemented as SQL views. In a controlled test, source changes made the incremental run read only 1 of 31,465 orders, 1 product and 2 customers. The indicators show net revenue of US$ 109.8 million, 86.2% of revenue in the Bikes category and strong monthly swings driven by the reseller channel.

Keywords: data warehouse; star schema; incremental ETL; staging area; Apache Airflow.

1 INTRODUÇÃO

Sistemas transacionais (*Online Transaction Processing* – OLTP) são projetados para registrar operações com rapidez e integridade, por isso armazenam dados em esquemas altamente normalizados. Essa organização é adequada ao registro de pedidos, mas dificulta consultas analíticas, que percorrem muitas tabelas e grandes volumes históricos. O *Data Warehouse* resolve esse problema ao integrar os dados em uma base orientada a assuntos, histórica e voltada à consulta (Inmon, 2005). A

modelagem dimensional de Kimball e Ross (2013) organiza o DW em tabelas fato, com as medidas de um processo de negócio, e dimensões, com o contexto dessas medidas; no modelo estrela, cada dimensão se liga diretamente à fato (Star..., 2026).

Alimentar o DW sem prejudicar o sistema de origem é parte do problema: em um ambiente real, a origem é o banco de produção. Por isso, este trabalho separa a leitura da origem, curta e restrita às mudanças, do processamento pesado, feito no próprio DW a partir de uma área de *stage*.

O objetivo é projetar e implementar um DW de vendas a partir do AdventureWorks2016 (Microsoft, 2026a), com análise do modelo OLTP, dez indicadores, modelo estrela em PostgreSQL, ETL incremental orquestrado pelo Apache Airflow e cálculo dos indicadores em SQL. O código-fonte está em https://github.com/lucaslnasc/adventureworks-dw (Adventureworks-dw..., 2026), e o texto segue o guia institucional (Centro Universitário Salesiano, 2024).

2 DESENVOLVIMENTO

O ambiente usa Docker Compose com quatro serviços: SQL Server 2022 com o AdventureWorks2016 (origem, que representa o banco de produção), PostgreSQL 17 (PostgreSQL Global Development Group, 2026) com a *stage* e o DW, Apache Airflow 3.3.2 e um PostgreSQL para os metadados do Airflow.

2.1 ANÁLISE DO MODELO TRANSACIONAL

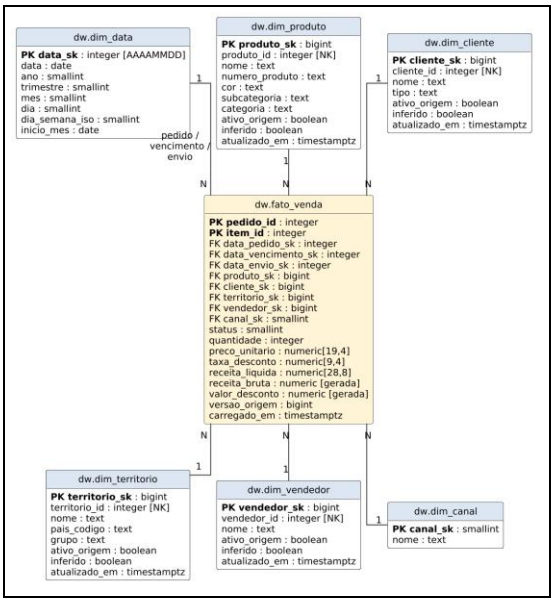
O AdventureWorks2016 representa uma fabricante fictícia de bicicletas, com cerca de 70 tabelas nos esquemas *Person*, *Sales*, *Production*, *Purchasing* e *HumanResources*, em terceira forma normal. O processo de negócio escolhido foi **vendas**, que envolve dez tabelas: SalesOrderHeader (31.465 pedidos), SalesOrderDetail (121.317 itens), Product, ProductSubcategory, ProductCategory, Customer, Person, Store, SalesTerritory e SalesPerson. No OLTP, saber a receita por categoria e território exige unir ao menos seis tabelas; o nome do cliente vem de Person ou de Store; o canal é apenas uma *flag* (OnlineOrderFlag); e pedidos online não têm vendedor.

2.2 MODELO DE DADOS

2.2.1 Modelo multidimensional proposto

O modelo segue o padrão estrela: a tabela fato dw.fato_venda liga-se diretamente a seis dimensões — dim_data, dim_produto, dim_cliente, dim_territorio, dim_vendedor e dim_canal —, sem relações entre dimensões (Figura 1). O script de criação está em sql/dw.sql.

Figura 1 – Modelo estrela do Data Warehouse



Fonte: elaboração própria (2026), gerado em PlantUML.

2.2.2 Granularidade da tabela fato

O grão da fato_venda é **um item de pedido de venda** (uma linha de SalesOrderDetail), o nível mais detalhado e o único em que o produto é conhecido. O número do pedido é mantido como dimensão degenerada e, com item_id, forma a chave primária, o que impede duplicações quando a carga é reexecutada. As medidas aditivas são quantidade, receita_bruta, valor_desconto e receita_liquida; preco_unitario e taxa_desconto são não aditivas. Receita bruta e desconto são colunas geradas pelo PostgreSQL, o que garante consistência entre as medidas.

2.2.3 Justificativa técnica das decisões de modelagem

Desnormalização. Subcategoria e categoria foram incorporadas à dim_produto, e Person e Store à dim_cliente, evitando o floco de neve (Kimball; Ross, 2013). **Chaves substitutas** (*_sk) isolam o DW das chaves do OLTP, preservadas em *_id. **Dimensão de data com múltiplos papéis** atende a pedido, vencimento e envio. **Membros "não informado"** (chave 0) substituem vendedor e território nulos, mantendo as chaves estrangeiras obrigatórias.

Dimensões de mudança lenta Tipo 1. As dimensões seguem o tipo 1 de *Slowly Changing Dimension* (SCD): são atualizadas por sobrescrita, pois os indicadores não exigem histórico. Registros excluídos na origem ficam com ativo_origem = false, sem quebrar as referências da fato. **Membros inferidos**: se uma venda chega antes da sua dimensão, a carga cria a linha com inferido = true, completada na carga seguinte da dimensão. **Integridade**: chaves estrangeiras, CHECK nas medidas e índices.

2.2.4 Dicionário de dados

O Quadro 1 resume as dimensões e o Quadro 2 descreve a fato; o dicionário completo, incluindo a *stage*, está em docs/dicionario_dados.md.

Quadro 1 – Resumo das dimensões

Dimensão	Principais atributos	Linhas
dim_data	data_sk (AAAAMMDD), data, ano, trimestre, mês, dia, dia da semana, início do mês	1.139
dim_produto	produto_id, nome, número, cor, subcategoria, categoria	504
dim_cliente	cliente_id, nome (pessoa ou loja), tipo	19.820
dim_territorio	territorio_id, nome, país, grupo geográfico	11
dim_vendedor	vendedor_id, nome	18
dim_canal	canal_sk (0 = Revendedor, 1 = Online), nome	2

Fonte: elaboração própria (2026). Nota: produto, cliente, território e vendedor têm ainda ativo_origem, inferido e atualizado_em.

Quadro 2 – Dicionário de dados da tabela dw.fato_venda

Coluna	Tipo	Descrição
pedido_id, item_id	integer	Chave primária composta; pedido_id é dimensão degenerada
data_pedido_sk, data_vencimento_sk, data_envio_sk	integer	Chaves para dim_data nos papéis de pedido, vencimento e envio (envio pode ser nulo)
produto_sk, cliente_sk, territorio_sk, vendedor_sk, canal_sk	bigint/smallint	Chaves substitutas para as demais dimensões
status	smallint	Situação do pedido (1 a 6; 5 = expedido)
quantidade, preco_unitario, taxa_desconto	integer, numeric	Unidades, preço de venda e desconto unitário (0 a 1)
receita_liquida	numeric(28,8)	Receita do item após desconto (LineTotal)
receita_bruta, valor_desconto	numeric (gerada)	quantidade × preço; receita bruta − receita líquida
versao_origem, carregado_em	bigint, timestamptz	Versão do Change Tracking e data da última gravação

Fonte: elaboração própria (2026).

2.3 DATA ETL

2.3.1 Área de stage e proteção da origem

Executar extração e transformação em uma só etapa manteria uma sessão aberta no banco de produção durante toda a rotina; por isso, o ETL foi dividido em duas etapas. Na **extração**, escrita em Python, cada grupo de tabelas lê da origem só as chaves alteradas, por chave primária, em uma transação *snapshot* — que não bloqueia quem grava — e copia as linhas para o esquema stage do PostgreSQL, ainda no formato do OLTP e com a operação (I, U ou D). Na **carga**, arquivos SQL executados no próprio DW aplicam as regras de negócio: nomes, tipos, chaves substitutas, SCD e exclusões. Assim, a origem fica livre ao fim da extração, e uma falha na carga é refeita a partir da *stage*, sem nova leitura da produção.

2.3.2 Estratégia de ETL incremental

As mudanças são detectadas pelo *Change Tracking* (CT) do SQL Server, que registra as chaves inseridas, alteradas ou excluídas, com um número de versão crescente (Microsoft, 2026b); ao contrário da comparação por ModifiedDate, ele também identifica exclusões. As dez tabelas foram divididas em **cinco grupos** — produto, cliente, território, vendedor e vendas —, cada um com seu próprio *checkpoint* em etl.controle. Sem *checkpoint*, o grupo faz carga completa; com ele, CHANGETABLE(CHANGES tabela, checkpoint) devolve só as chaves alteradas. Mudanças indiretas são propagadas: alterar uma pessoa atualiza o cliente e o vendedor dela. O Quadro 3 mostra a extração do grupo cliente.

Quadro 3 – Consulta de extração incremental do grupo cliente (origem → stage)

```
-- 1. Chaves afetadas desde o checkpoint (cliente, pessoa ou loja alterados)
SELECT id, MIN(op) op INTO #chaves FROM (
    SELECT ct.CustomerID id, ct.SYS_CHANGE_OPERATION op
    FROM CHANGETABLE(CHANGES Sales.Customer, @checkpoint) ct
    UNION ALL SELECT c.CustomerID, 'U' FROM Sales.Customer c
    JOIN CHANGETABLE(CHANGES Person.Person, @checkpoint) ct
      ON ct.BusinessEntityID = c.PersonID
    UNION ALL SELECT c.CustomerID, 'U' FROM Sales.Customer c
    JOIN CHANGETABLE(CHANGES Sales.Store, @checkpoint) ct
      ON ct.BusinessEntityID = c.StoreID) x
GROUP BY id;
-- 2. Linhas atuais dessas chaves (ausente na origem = exclusão)
SELECT CASE WHEN c.CustomerID IS NULL THEN 'D' WHEN k.op = 'I' THEN 'I' ELSE 'U' END
operacao,
       k.id customer_id,
       c.PersonID person_id, c.StoreID store_id, p.FirstName first_name,
       p.MiddleName middle_name, p.LastName last_name, s.Name store_name
FROM #chaves k LEFT JOIN Sales.Customer c ON c.CustomerID = k.id
LEFT JOIN Person.Person p ON p.BusinessEntityID = c.PersonID
LEFT JOIN Sales.Store s ON s.BusinessEntityID = c.StoreID;
```

Fonte: elaboração própria (2026), arquivo etl/origem.py do repositório.

Cada carga é um arquivo SQL executado em **uma única transação**, que termina chamando etl.registrar_carga: essa função grava a auditoria em etl.carga e avança o *checkpoint* do grupo para a versão lida na extração. Se algo falhar, nada é gravado e a execução seguinte relê as mesmas mudanças. O Quadro 4 mostra a carga da dim_cliente, e o Quadro 5, os passos centrais da carga da fato (que também aplica as exclusões de pedidos e itens).

Quadro 4 – Carga da dimensão de cliente (stage → DW, SCD Tipo 1)

```
-- exclusões na origem: a linha fica inativa
UPDATE dw.dim_cliente d SET ativo_origem = false, atualizado_em = now()
FROM stage.cliente s WHERE s.operacao = 'D' AND s.customer_id = d.cliente_id;
WITH gravadas AS (
  INSERT INTO dw.dim_cliente AS d (cliente_id, nome, tipo)
  SELECT customer_id,
        COALESCE(store_name,
              NULLIF(btrim(concat_ws(' ', first_name, middle_name, last_name)), ''),
              'Não informado'),
        CASE WHEN store_id IS NOT NULL THEN 'Loja'
            WHEN person_id IS NOT NULL THEN 'Pessoa' ELSE 'Não informado' END
  FROM stage.cliente WHERE operacao <> 'D'
  ON CONFLICT (cliente_id) DO UPDATE SET nome = EXCLUDED.nome, tipo = EXCLUDED.tipo,
      ativo_origem = true, inferido = false, atualizado_em = now()
  WHERE NOT d.ativo_origem OR d.inferido
     OR (d.nome, d.tipo) IS DISTINCT FROM (EXCLUDED.nome, EXCLUDED.tipo)
  RETURNING 1)
-- auditoria e checkpoint do grupo, na mesma transação
SELECT etl.registrar_carga('cliente', count(*)::integer) FROM gravadas;
```

Fonte: elaboração própria (2026), arquivo sql/carga/dim_cliente.sql do repositório.

Quadro 5 – Passos centrais da carga da fato (stage → DW)

```
-- membro inferido: produto que chegou na venda antes da dimensão
INSERT INTO dw.dim_produto (produto_id, nome, numero_produto, subcategoria, categoria,
inferido)
SELECT DISTINCT product_id, 'Não informado', 'N/I', 'Não informado', 'Não informado', true
FROM stage.item WHERE operacao <> 'D' ON CONFLICT (produto_id) DO NOTHING;
-- itens novos ou alterados (pedido_dw = cabeçalhos com as chaves do DW)
WITH gravadas AS (
  INSERT INTO dw.fato_venda AS f (pedido_id, item_id, data_pedido_sk, ..., receita_liquida)
  SELECT i.sales_order_id, i.sales_order_detail_id, p.data_pedido_sk, ..., i.line_total
  FROM stage.item i JOIN pedido_dw p ON p.pedido_id = i.sales_order_id
  JOIN dw.dim_produto pr ON pr.produto_id = i.product_id
  WHERE i.operacao <> 'D'
  ON CONFLICT (pedido_id, item_id) DO UPDATE SET quantidade = EXCLUDED.quantidade, ...
  WHERE (f.quantidade, ...) IS DISTINCT FROM (EXCLUDED.quantidade, ...)
  RETURNING 1)
SELECT etl.registrar_carga('vendas', count(*)::integer) FROM gravadas;
```

Fonte: elaboração própria (2026), adaptado de sql/carga/fato_venda.sql do repositório.

2.3.3 Fluxo da pipeline no Apache Airflow

No Airflow, a pipeline é um *Directed Acyclic Graph* (DAG), isto é, um grafo acíclico dirigido de tarefas (Apache Software Foundation, 2026). A DAG adventureworks_simples roda diariamente (@daily), com uma execução ativa por vez, até quatro tarefas simultâneas, duas novas tentativas por tarefa e limite de 15 minutos por tarefa. As 17 tarefas estão em seis blocos (TaskGroup), descritos no Quadro 6. As extrações usam o *pool* origem_sqlserver, de duas vagas, que limita a duas as conexões simultâneas à produção. Cargas e validações usam o SQLExecuteQueryOperator, que executa os arquivos de sql/ no DW.

Quadro 6 – Blocos e tarefas da DAG adventureworks_simples

Bloco	Tarefas	Função
preparacao	verificar_dw, verificar_origem	Confere DW, CT e checkpoints
extracao	extrair_produto, extrair_cliente, extrair_territorio, extrair_vendedor, extrair_vendas	Origem → stage; únicas que leem a produção
dimensoes	carregar_dim_produto, carregar_dim_cliente, carregar_dim_territorio, carregar_dim_vendedor, carregar_dim_data	Stage → dimensões, logo após a sua extração
fato	carregar_fato_venda	Stage → fato
qualidade	validar_cabecalhos, validar_datas, validar_checkpoints	Validações em paralelo
(final)	gerar_evidencias	Indicadores e auditoria em arquivo

Fonte: elaboração própria (2026).

As dependências formam um leque, não uma fila: cada dimensão depende só da sua extração, a fato espera todas as dimensões e, se uma tarefa falha, só ela é repetida. A Figura 2 mostra uma execução de 13 segundos.

Figura 2 – Grafo da DAG adventureworks_simples com os blocos expandidos



Fonte: captura de tela do Apache Airflow, elaboração própria (2026).

2.3.4 Comprovação da carga incremental

Na origem, foram alterados um item do pedido 43659, a cor do produto 776 e o sobrenome do cliente 11000; a DAG rodou, os valores foram restaurados e ela rodou de novo. A Tabela 1 resume etl.carga.

Tabela 1 – Linhas por grupo em cada execução (stage / gravadas)

Execução	Produto	Cliente	Território	Vendedor	Vendas
Carga inicial (completa)	504 / 504	19.820 / 19.820	10 / 10	17 / 17	152.782 / 121.317
Sem alterações	0 / 0	0 / 0	0 / 0	0 / 0	0 / 0
Após alteração	1 / 1	2 / 1	0 / 0	0 / 0	2 / 1
Após reversão	1 / 1	2 / 1	0 / 0	0 / 0	2 / 1

Fonte: elaboração própria (2026), a partir de etl.carga. Nota: em vendas, stage = pedidos + itens.

A *stage* de cliente recebeu também o cliente do pedido 43659, cuja pessoa é atualizada por um *trigger* da origem; como o nome dele não mudou, não foi regravado. Após a reversão, os indicadores voltaram aos valores originais.

2.4 INDICADORES

Os dez indicadores cobrem faturamento, volume, clientes, evolução temporal e nível de serviço. Cada um é uma *view* do esquema dw, criada no script sql/kpis.sql a partir de um SELECT direto sobre a tabela fato e, quando necessário, as dimensões. Todos consideram apenas pedidos expedidos (status = 5), e os valores são comerciais, sem impostos e frete. O Quadro 7 mostra as tabelas e os campos usados por indicador; os Quadros 8 a 11 trazem as consultas completas.

Quadro 7 – Tabelas e campos utilizados por indicador

Nº	Indicador	Tabelas	Campos utilizados	Cálculo
1	Receita bruta	fato_venda	receita_bruta, status	Soma
2	Descontos	fato_venda	valor_desconto, status	Soma
3	Receita líquida	fato_venda	receita_liquida, status	Soma
4	Unidades	fato_venda	quantidade, status	Soma
5	Pedidos	fato_venda	pedido_id, status	Conta distinta
6	Ticket médio	fato_venda	receita_liquida, pedido_id, status	Receita ÷ pedidos
7	Taxa de desconto	fato_venda	valor_desconto, receita_bruta, status	Desconto ÷ receita bruta
8	Clientes ativos	fato_venda, dim_cliente	cliente_sk, status; cliente_id	Conta distinta
9	Crescimento mensal	fato_venda, dim_data	receita_liquida, data_pedido_sk, status; data_sk, data, inicio_mes	Variação sobre o mês anterior
10	Expedição no prazo	fato_venda, dim_data (2×)	pedido_id, data_vencimento_sk, data_envio_sk, status; data_sk, data	No prazo ÷ com envio

Fonte: elaboração própria (2026). Nota: campos antes do ponto e vírgula são da fato; os demais, da dimensão. Percentuais multiplicados por 100.

Quadro 8 – Consultas SQL dos indicadores 1 a 4

```
-- 1. Receita bruta (view dw.kpi_01_receita_bruta)
SELECT COALESCE(SUM(f.receita_bruta), 0) AS receita_bruta_um
FROM dw.fato_venda f WHERE f.status = 5;
-- 2. Descontos concedidos (view dw.kpi_02_descontos)
SELECT COALESCE(SUM(f.valor_desconto), 0) AS descontos_um
FROM dw.fato_venda f WHERE f.status = 5;
-- 3. Receita líquida (view dw.kpi_03_receita_liquida)
SELECT COALESCE(SUM(f.receita_liquida), 0) AS receita_liquida_um
FROM dw.fato_venda f WHERE f.status = 5;
-- 4. Unidades vendidas (view dw.kpi_04_unidades)
SELECT COALESCE(SUM(f.quantidade), 0) AS unidades
FROM dw.fato_venda f WHERE f.status = 5;
```

Fonte: elaboração própria (2026), arquivo sql/kpis.sql do repositório.

Quadro 9 – Consultas SQL dos indicadores 5 a 8

```
-- 5. Pedidos expedidos (view dw.kpi_05_pedidos)
SELECT COUNT(DISTINCT f.pedido_id) AS pedidos
FROM dw.fato_venda f WHERE f.status = 5;
-- 6. Ticket médio (view dw.kpi_06_ticket_medio)
SELECT SUM(f.receita_liquida) / NULLIF(COUNT(DISTINCT f.pedido_id), 0) AS ticket_medio_um
FROM dw.fato_venda f WHERE f.status = 5;
-- 7. Taxa de desconto ponderada (view dw.kpi_07_taxa_desconto)
SELECT 100.0 * SUM(f.valor_desconto) / NULLIF(SUM(f.receita_bruta), 0)
      AS desconto_ponderado_pct
FROM dw.fato_venda f WHERE f.status = 5;
-- 8. Clientes com compra (view dw.kpi_08_clientes_ativos)
SELECT COUNT(DISTINCT c.cliente_id) AS clientes_com_compra
FROM dw.fato_venda f
JOIN dw.dim_cliente c ON c.cliente_sk = f.cliente_sk WHERE f.status = 5;
```

Fonte: elaboração própria (2026), arquivo sql/kpis.sql do repositório.

O crescimento mensal (Quadro 10) monta um calendário com todos os meses do período, para que meses sem venda apareçam com receita zero, e compara cada mês com o anterior pela função de janela LAG. A expedição no prazo (Quadro 11) usa a dim_data duas vezes, nos papéis de vencimento e de envio, e conta cada pedido uma única vez.

Quadro 10 – Consulta SQL do indicador 9 – crescimento mensal da receita

```
-- 9. Crescimento mensal (view dw.kpi_09_crescimento_mensal)
WITH limites AS (
    SELECT date_trunc('month', MIN(d.data)) AS inicio,
           date_trunc('month', MAX(d.data)) AS fim
    FROM dw.fato_venda f
    JOIN dw.dim_data d ON d.data_sk = f.data_pedido_sk
), calendario AS (
    SELECT generate_series(inicio, fim, interval '1 month')::date AS mes FROM limites
), vendas AS (
    SELECT d.inicio_mes AS mes, SUM(f.receita_liquida) AS receita
    FROM dw.fato_venda f
    JOIN dw.dim_data d ON d.data_sk = f.data_pedido_sk
    WHERE f.status = 5
    GROUP BY d.inicio_mes
), mensal AS (
    SELECT c.mes, COALESCE(v.receita, 0) AS receita
    FROM calendario c
    LEFT JOIN vendas v ON v.mes = c.mes
)
SELECT mes, receita,
       LAG(receita) OVER w AS receita_anterior,
       100.0 * (receita - LAG(receita) OVER w)
          / NULLIF(LAG(receita) OVER w, 0) AS crescimento_pct
FROM mensal
WINDOW w AS (ORDER BY mes);
```

Fonte: elaboração própria (2026), arquivo sql/kpis.sql do repositório.

Quadro 11 – Consulta SQL do indicador 10 – expedição no prazo

```
-- 10. Expedição no prazo (view dw.kpi_10_expedicao_no_prazo)
WITH pedidos AS (
    SELECT DISTINCT f.pedido_id, e.data AS data_envio, v.data AS data_vencimento
    FROM dw.fato_venda f
    JOIN dw.dim_data v ON v.data_sk = f.data_vencimento_sk
    LEFT JOIN dw.dim_data e ON e.data_sk = f.data_envio_sk
    WHERE f.status = 5
)
SELECT COUNT(*) FILTER (WHERE data_envio IS NOT NULL) AS pedidos_com_data,
       COUNT(*) FILTER (WHERE data_envio IS NULL) AS pedidos_sem_data,
       COUNT(*) FILTER (WHERE data_envio <= data_vencimento) AS pedidos_no_prazo,
       100.0 * COUNT(*) FILTER (WHERE data_envio <= data_vencimento)
          / NULLIF(COUNT(*) FILTER (WHERE data_envio IS NOT NULL), 0)
          AS expedicao_no_prazo_pct
FROM pedidos;
```

Fonte: elaboração própria (2026), arquivo sql/kpis.sql do repositório.

2.5 RESULTADOS DOS INDICADORES

A Tabela 2 apresenta o resultado de cada consulta para os pedidos de 31 de maio de 2011 a 30 de junho de 2014, em dólares americanos, moeda do AdventureWorks.

Tabela 2 – Resultados dos indicadores

Nº	Indicador	Resultado
1	Receita bruta	US$ 110.373.889,31
2	Descontos	US$ 527.507,91
3	Receita líquida	US$ 109.846.381,40
4	Unidades	274.914
5	Pedidos	31.465
6	Ticket médio	US$ 3.491,07
7	Taxa de desconto ponderada	0,48%
8	Clientes com compra	19.119 (96,5% dos 19.820 cadastrados)
9	Crescimento mensal	de −99,09% a +813,96% (38 meses)
10	Expedição no prazo	100% (31.465 pedidos)

Fonte: elaboração própria (2026), a partir das views do esquema dw.

Para interpretar os totais, os indicadores foram cortados pelas dimensões com as consultas do Quadro 12 (arquivo sql/analises.sql), que juntam a fato à dim_produto, à dim_canal e à dim_data. As Tabelas 3 e 4 mostram os resultados por categoria e por canal.

Quadro 12 – Consultas SQL de análise por dimensão

```
-- A1. Receita e unidades por categoria de produto
SELECT p.categoria, SUM(f.receita_liquida) AS receita,
      ROUND(100 * SUM(f.receita_liquida) / SUM(SUM(f.receita_liquida)) OVER (), 1)
         AS pct_receita,
      SUM(f.quantidade) AS unidades
FROM dw.fato_venda f
JOIN dw.dim_produto p ON p.produto_sk = f.produto_sk
WHERE f.status = 5
GROUP BY p.categoria ORDER BY receita DESC;
-- A2. Receita, pedidos e ticket médio por canal
SELECT c.nome AS canal, SUM(f.receita_liquida) AS receita,
      COUNT(DISTINCT f.pedido_id) AS pedidos,
      SUM(f.receita_liquida) / COUNT(DISTINCT f.pedido_id) AS ticket_medio
FROM dw.fato_venda f
JOIN dw.dim_canal c ON c.canal_sk = f.canal_sk
WHERE f.status = 5
GROUP BY c.nome ORDER BY receita DESC;
-- A3. Pedidos e receita por ano e canal
SELECT d.ano, c.nome AS canal, COUNT(DISTINCT f.pedido_id) AS pedidos,
      SUM(f.receita_liquida) AS receita
FROM dw.fato_venda f
JOIN dw.dim_data d ON d.data_sk = f.data_pedido_sk
JOIN dw.dim_canal c ON c.canal_sk = f.canal_sk
WHERE f.status = 5
GROUP BY d.ano, c.nome ORDER BY d.ano, canal;
```

Fonte: elaboração própria (2026), arquivo sql/analises.sql do repositório.

Tabela 3 – Receita e unidades por categoria (consulta A1)

Categoria	Receita líquida	% da receita	Unidades
Bikes	US$ 94.651.172,70	86,2%	90.268
Components	US$ 11.802.593,29	10,7%	49.044
Clothing	US$ 2.120.542,52	1,9%	73.670
Accessories	US$ 1.272.072,88	1,2%	61.932

Fonte: elaboração própria (2026), a partir da consulta A1.

Tabela 4 – Receita, pedidos e ticket médio por canal (consulta A2)

Canal	Receita líquida	Pedidos	Ticket médio
Revendedor	US$ 80.487.704,18	3.806	US$ 21.147,58
Online	US$ 29.358.677,22	27.659	US$ 1.061,45

Fonte: elaboração própria (2026), a partir da consulta A2.

Os resultados revelam padrões: a) *Bikes* responde por 86,2% da receita, mas roupas e acessórios somam quase metade das unidades (Tabela 3); b) revendedores geram 73,3% da receita com ticket vinte vezes maior que o online, diferença que o ticket médio geral esconde (Tabela 4); c) as oscilações mensais, como +813,96% em outubro de 2011, vêm dos revendedores, cujos pedidos se concentram em alguns meses, e a queda de −99,09% em junho de 2014 reflete só o fim dos dados; d) pela consulta A3, os pedidos online saltaram de 2.743 em 2012 para 12.584 em 2013, puxando o crescimento de 30,1% da receita; e) todos os pedidos foram enviados em sete dias, com vencimento em doze, o que explica os 100% no prazo.

3 CONSIDERAÇÕES FINAIS

O trabalho construiu um *Data Warehouse* de vendas em modelo estrela, com dez indicadores em SQL. A área de *stage* separou a leitura da origem, restrita às chaves alteradas e a duas conexões simultâneas, do processamento pesado, feito no DW; assim, o banco de produção não fica preso à rotina. A DAG em blocos paralelos, com *checkpoint* por grupo e membros inferidos, repete só a etapa que falhou.

Como limitações, as dimensões não guardam histórico e a retenção do *Change Tracking* é de sete dias; como trabalhos futuros, sugerem-se SCD Tipo 2, margem de lucro e os processos de compras e estoque.

REFERÊNCIAS

ADVENTUREWORKS-DW: data warehouse AdventureWorks com Apache Airflow. [S. l.]: GitHub, 2026. Repositório de código-fonte. Disponível em: https://github.com/lucaslnasc/adventureworks-dw. Acesso em: 4 nov. 2026.

APACHE SOFTWARE FOUNDATION. **Apache Airflow documentation**: version 3.3.2. [S. l.], 2026. Disponível em: https://airflow.apache.org/docs/apache-airflow/3.3.2/. Acesso em: 4 nov. 2026.

CENTRO UNIVERSITÁRIO SALESIANO. **Guia de elaboração e normalização de trabalhos acadêmicos e de pesquisa**. Vitória, 2024. Disponível em: https://unisales.br/wp-content/uploads/2024/07/NOVO-GUIA-DE-ELABORACAO-E-NORMALIZACAO-DE-TRABALHOS-ACADEMICOS-E-DE-PESQUISA-29.05.pdf. Acesso em: 4 nov. 2026.

INMON, William H. **Building the data warehouse**. 4. ed. Indianapolis: Wiley, 2005.

KIMBALL, Ralph; ROSS, Margy. **The data warehouse toolkit**: the definitive guide to dimensional modeling. 3. ed. Indianapolis: Wiley, 2013.

MICROSOFT. **Bancos de dados de exemplo AdventureWorks**. [S. l.], 2026a. Disponível em: https://learn.microsoft.com/pt-br/sql/samples/adventureworks-install-configure. Acesso em: 4 nov. 2026.

MICROSOFT. **Sobre o controle de alterações (SQL Server)**. [S. l.], 2026b. Disponível em: https://learn.microsoft.com/pt-br/sql/relational-databases/track-changes/about-change-tracking-sql-server. Acesso em: 4 nov. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 17 documentation**. [S. l.], 2026. Disponível em: https://www.postgresql.org/docs/17/. Acesso em: 4 nov. 2026.

STAR schema. In: WIKIPEDIA: the free encyclopedia. [S. l.], 2026. Disponível em: https://en.wikipedia.org/wiki/Star_schema. Acesso em: 4 nov. 2026.